



Flood Probability Classifier

Gradient Boosting Machine Model Documentation

MKM Research Labs

Version 2.0 — 11-March-2026

David K Kelly

CONFIDENTIAL — PROPRIETARY

Legal Notice

PROPRIETARY AND CONFIDENTIAL

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs (“MKM”).

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an “as is” basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

All rights reserved. © 2021–2026 MKM Research Labs.

Governed by the laws of the United Kingdom.

Contents

1	Executive Summary	3
2	Model Purpose and Scope	3
2.1	Purpose	3
2.2	Scope	3
2.3	Model Chain Position	4
3	Mathematical Framework	4
3.1	Model Family	4
3.2	Feature Space	4
3.3	Physical Motivation for Log Transformation	5
3.4	Rising vs Falling Water	5
3.5	Target Variable	5
3.6	Training Data	5
3.6.1	v2.0 (stressm): Multi-Storm Sequence Training	5
3.6.2	v1.3 (stress): Single-Storm Balanced Training	6
4	Hyperparameters	6
5	Implementation Details	7
5.1	Source Code	7
5.2	Training Pipelines	7
5.2.1	v2.0 stressm Pipeline	7
5.2.2	v1.3 stress Pipeline (retained for single-storm Tab 7)	7
5.3	Prediction Interface	7
5.4	Probability Surface	7
5.5	Serialisation	8
6	Validation and Metrics	8
6.1	Evaluation Metrics	8
6.2	Production Results (v2.0 stressm)	9
6.3	Version Comparison: v1.1 → v1.2 → v1.3 → v2.0	9
6.3.1	Asymmetry Evidence: Rising vs Falling Water	9
6.4	Feature Importance	10
6.5	Automated Test Results	10
6.6	Test Document History	14
7	Sensitivity Analysis	15
8	Model Limitations and Known Weaknesses	16
9	Change History	17

1 Executive Summary

This document presents the mathematical framework, implementation, and validation of the Flood Probability Classifier (MKM-FC-001), a Gradient Boosting Machine (GBM) model that estimates per-gauge flood probability as a function of log-transformed water level, storm time, and hydrograph derivatives: $F(\ln \frac{w}{s}, \ln \frac{t+1}{T}, \Delta \ln \frac{w}{s}, \Delta^2 \ln \frac{w}{s}) \rightarrow P(\text{flood})$, where s is the severe flood threshold and $T = 168$ hours.

The classifier is trained per gauge using stress test vectors derived from historical storm simulations. All features are log-transformed relative to gauge-specific thresholds, producing a scale-invariant representation that generalises across gauges with different flood severity levels.

v1.3 enhancement introduced four log-transformed features: $\ln(w/s)$, $\ln((t+1)/168)$, $\Delta \ln(w/s)$, and $\Delta^2 \ln(w/s)$. This produced a mean AUC of 0.9525 across 40 gauges, a 3.2% improvement over v1.2.

v2.0 upgrade (multi-storm): Version 2.0 replaces the single-storm training corpus with sequences generated by the Storm Sequence Generator (MKM-SS-001). Instead of 1,000 independent storms ($\times 168\text{h} = 168,000$ vectors per gauge), the classifier now trains on **10,000 multi-storm sequences** ($\times 168\text{h} = 1,680,000$ vectors per gauge)—ten times the data. Crucially, multi-storm hydrographs contain *multi-peak* shapes driven by compounding antecedent-wetness effects that single-storm data cannot represent. The feature space and hyperparameters are unchanged from v1.3; the gains come entirely from richer, larger training data.

The mean AUC-ROC across all 52 Thames gauges improved from 0.9525 (v1.3) to ≈ 0.993 (v2.0), a further 4.2% improvement. The stressm classifiers are stored in `data/output/stressm/` and trained via `app.py port --stressm --train-classifier`.

Key risk: The model is trained on synthetically noised data (time-decaying Gaussian perturbation), which creates intentional early-hour uncertainty. The derivative features are computed from the log-transformed noised hydrograph, preserving the uncertainty structure in the velocity and acceleration signals.

2 Model Purpose and Scope

2.1 Purpose

The Flood Probability Classifier serves two functions within the platform:

- **Stress testing:** Given a stress storm scenario, estimate the probability of flood alert breach at each gauge as a function of evolving water levels during the storm event.
- **P&L estimation:** Convert flood probabilities into cash-equivalent stress P&L for PRS trade positions, using the CDS-in-stress analogy: $\text{cash_price} = \text{notional} \times P(\text{flood}) \times \text{direction}$.

2.2 Scope

- **In scope:** Per-gauge binary classification (flood/no-flood) using water level, temporal, and hydrograph derivative features.
- **Out of scope:** Multi-gauge spatial correlation, flood severity estimation (depth/velocity), and climate change projection.

2.3 Model Chain Position

The classifier now sits downstream of the Storm Sequence Generator:

MKM-SI-001 → **MKM-SS-001** (sequences) → MKM-SG-001 (hydrograph) → **MKM-FC-001** (v2.0) → Stress F

For single-storm stress testing (Trading Desk Tab 7), the v1.3 classifiers stored in `data/output/stress/` remain in use. The v2.0 stressm classifiers stored in `data/output/stressm/` power the Portfolio Stress tab (Tab 8).

3 Mathematical Framework

3.1 Model Family

The model uses `sklearn.ensemble.GradientBoostingClassifier`, which constructs an additive ensemble of regression trees optimised under the binomial deviance loss:

$$\mathcal{L}(y, F) = - \sum_{i=1}^N [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

where $p_i = \sigma(F(x_i))$ and σ is the logistic function. The model is built sequentially:

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)$$

where h_m is a regression tree fit to the negative gradient of the loss, and ν is the learning rate.

3.2 Feature Space

The model uses a four-dimensional log-transformed feature space:

Feature	Type	Description
<code>log_h_s</code>	Continuous	$\ln(\tilde{w}(t)/s)$; water level relative to severe threshold s . Zero at severe level, negative below, positive above.
<code>log_t_end</code>	Continuous	$\ln((t+1)/168)$; log storm time. Ranges from ≈ -5.1 (hour 0) to 0 (hour 167).
<code>delta_log_h</code>	Continuous	$\Delta \ln(w/s)(t) = \text{log_h_s}(t) - \text{log_h_s}(t-1)$; log velocity (positive rising, negative falling).
<code>delta2_log_h</code>	Continuous	$\Delta^2 \ln(w/s)(t) = \text{delta_log_h}(t) - \text{delta_log_h}(t-1)$; log acceleration.

All derivative features are computed from the log-transformed noised hydrograph $\ln(\tilde{w}(t)/s)$, not the clean signal. This ensures the model learns to interpret derivative signals that include the same observation noise present in real-time monitoring. At prediction time, raw derivatives Δw and $\Delta^2 w$ are passed in and converted to log-space internally by the predictor.

3.3 Physical Motivation for Log Transformation

The raw-level feature space has two limitations:

1. **Gauge heterogeneity:** Thames gauges have severe thresholds ranging from approximately 3 m to 7 m AOD. A water level of 4.5 m carries very different flood implications depending on the gauge’s threshold. The log transform $\ln(w/s)$ normalises to a common scale: $\ln(w/s) = 0$ always means “at severe threshold”, regardless of absolute level.
2. **Velocity interpretation:** A rise of 0.3 m/h from 1 m is alarming; the same rise from 5 m is less informative. Log-velocity $\Delta \ln(w/s) \approx \Delta w/w$ normalises by the current level, providing a relative rate of change. This is analogous to using log-returns in financial risk models.

The time feature $\ln((t+1)/168)$ is restored from v1.2 in log scale. This compresses the early storm hours (where uncertainty is highest) and stretches the late hours (where remaining uncertainty is lower), matching the structure of the noise model $\sigma(t)$.

3.4 Rising vs Falling Water

The log-velocity feature $\Delta \ln(w/s)$ preserves the directional asymmetry introduced in v1.2:

- **Rising water** ($\Delta \ln(w/s) > 0$): The storm has not yet peaked. $P(\text{flood})$ should reflect uncertainty about the eventual peak.
- **Falling water** ($\Delta \ln(w/s) < 0$): The storm has already peaked. If near alert level, the peak was likely above it. $P(\text{flood})$ should be higher, reflecting stronger evidence.

The log-acceleration $\Delta^2 \ln(w/s)$ further discriminates between a decelerating rise (approaching peak) and an accelerating rise (early surge).

3.5 Target Variable

Binary label $y \in \{0, 1\}$ indicating whether the storm breaches the gauge’s flood alert trigger level at any point during the event. The label is computed from the stress test vector:

$$y_i = \begin{cases} 1 & \text{if } \max_t w_i(t) \geq \text{alert_level} \\ 0 & \text{otherwise} \end{cases}$$

3.6 Training Data

3.6.1 v2.0 (stressm): Multi-Storm Sequence Training

Version 2.0 trains on the full catalogue of 10,000 `StormSequence` objects generated by MKM-SS-001 (Section 2.3). Each sequence contains 1–5 individual storms with calibrated durations, inter-storm gaps, and correlated intensity factors. The training corpus is therefore the complete population, not a balanced sample.

For each sequence, a multi-peak hydrograph is synthesised at each gauge by compositing the individual storm responses in temporal order. Each storm contributes a water level arc derived from its intensity factor and the gauge’s spatial response parameters (`base_level_m`, `level_change_m`, `peak_position`). Between storms, the water level decays toward baseline at the catchment drainage rate, modelling the partial recovery that determines antecedent wetness for the next storm.

The same time-decaying Gaussian noise is applied per storm arc:

$$\tilde{w}(t) = w(t) + \varepsilon(t), \quad \varepsilon(t) \sim \mathcal{N}(0, \sigma(t)^2)$$

$$\sigma(t) = 0.5 \cdot e^{-3t/167} + 0.1$$

The feature extraction is identical to v1.3; the features are computed from the full 168-hour noised multi-peak hydrograph:

$$\mathbf{x}_i = [\ell(t), \tau(t), \Delta\ell(t), \Delta^2\ell(t)] \in \mathbb{R}^4$$

Total training corpus: **1,680,000 vectors per gauge** (10,000 sequences $\times$ 168 hours).

3.6.2 v1.3 (stress): Single-Storm Balanced Training

For reference, the v1.3 (single-storm) training pipeline used a balanced sample of 1,000 storms from the 10,000-storm catalogue:

- **Alert storms (260):** All storms that breached the flood alert trigger, sourced from `stress_storms.json`.
- **Non-alert storms (740):** Randomly sampled from the non-alert population.

Total: 168,000 vectors per gauge (1,000 storms $\times$ 168 hours).

Why v2.0 does not need balanced sampling: The multi-storm sequences naturally contain diverse hydrograph shapes including: (i) sequences where the gauge barely rises (Storm 1 is weak, far away); (ii) sequences where Storm 1 causes a modest rise and Storm 2 produces the flood (antecedent-wetness amplification); (iii) catastrophic sequences where all storms breach the severe threshold. This diversity provides the classifier with sufficient negative examples without artificial sub-sampling.

Log-level features are computed identically to v1.3:

$$\ell(t) = \ln\left(\max\left(\frac{\tilde{w}(t)}{s}, 10^{-8}\right)\right), \quad \Delta\ell(t) = \ell(t) - \ell(t-1), \quad \Delta^2\ell(t) = \Delta\ell(t) - \Delta\ell(t-1)$$

$$\tau(t) = \ln\left(\frac{t+1}{168}\right)$$

with boundary conditions $\Delta\ell(0) = \Delta^2\ell(0) = 0$.

4 Hyperparameters

Parameter	Value	Rationale
<code>n_estimators</code>	200	Sufficient for convergence without overfitting
<code>max_depth</code>	4	Captures water-level $\times$ hour interaction
<code>learning_rate</code>	0.1	Standard shrinkage for 200 rounds
<code>subsample</code>	0.8	Row sampling to reduce variance
<code>min_samples_leaf</code>	20	Prevents small-node overfitting
<code>random_state</code>	42	Reproducibility
<code>test_size</code>	0.2	80/20 train-test split

Stratified train-test split ensures the class ratio is preserved in both partitions.

5 Implementation Details

5.1 Source Code

- **Primary module:** `src/models/stress/flood_classifier.py`
- **Training class:** `FloodClassifierTrainer`
- **Prediction class:** `FloodPredictor`

5.2 Training Pipelines

5.2.1 v2.0 stressm Pipeline

1. Generate 10,000 `StormSequence` objects via MKM-SS-001 (`--stressm`)
2. For each gauge, synthesise 168-hour multi-peak hydrograph per sequence
3. Apply time-decaying noise; compute 4 log-transformed feature vectors
4. Build feature matrix $\mathbf{X} \in \mathbb{R}^{1,680,000 \times 4}$ and label vector $\mathbf{y} \in \{0, 1\}^{1,680,000}$
5. Skip single-class gauges
6. Stratified 80/20 train-test split with `random_state=42`
7. Fit `GradientBoostingClassifier` on training set
8. Evaluate on test set: accuracy, AUC-ROC, Brier score, log loss
9. Serialise model to `data/output/stressm/GAUGE-{id}.joblib`
10. Write summary to `data/output/stressm/training_summary.json`

5.2.2 v1.3 stress Pipeline (retained for single-storm Tab 7)

1. Load stress test vectors from `data/input/thames/stress_vectors.json`
2. For each gauge, extract feature matrix $\mathbf{X} \in \mathbb{R}^{168,000 \times 4}$ and label vector $\mathbf{y} \in \{0, 1\}^{168,000}$
3. Stratified 80/20 split; fit; evaluate; serialise to `data/output/stress/GAUGE-{id}.joblib`

5.3 Prediction Interface

Three prediction modes are provided:

- `predict(gauge_id, water_level, hour, delta_w, delta2_w)` — Single-point $\hat{p}$; `delta_w` and `delta2_w` default to 0.0 (steady state) for backward compatibility
- `predict_series(gauge_id, levels, hours, delta_ws, delta2_ws)` — Vectorised time series
- `predict_surface(gauge_id, ..., delta_w, delta2_w)` — Full (w, t) probability grid at constant derivatives (default 0.0)

5.4 Probability Surface

The `predict_surface` mode produces a two-dimensional grid of flood probabilities $P(w, t)$ over a range of water levels w and storm hours t . This is exposed in the stress test UI as the “Surface” tab, displaying a heat-map table with the following structure:

Axis	Description
Rows (Y)	Water levels in 0.5 m steps, descending from highest to lowest
Columns (X)	Storm hours at 4-hour intervals: H0, H4, H8, ..., H56
Cells	$P(\text{flood})$ as a percentage to one decimal place

The surface grid is computed on-demand during stress scenario execution with $\Delta w = 0$ and $\Delta^2 w = 0$ (steady-state assumption), representing the probability at each grid point if the water level were neither rising nor falling. The hourly time series uses actual derivatives computed from the hydrograph. Water level bounds are derived from the storm hydrograph range, extended to cover the gauge's full trigger band (alert through severe + 0.5 m buffer).

Visual encoding:

- Cells below the alert level: neutral (white) background with grey intensity shading proportional to $P(\text{flood})$.
- Cells in the alert band ($\text{alert} \leq w < \text{warning}$): amber background (`#FFF8E1`).
- Cells in the warning band ($\text{warning} \leq w < \text{severe}$): orange background (`#FFF3E0`).
- Cells at or above the severe level: red background (`#FFE0E0`).
- Post knock-out (after the first trigger breach hour): cells are left blank with a grey background, reflecting contract termination.

Within each trigger band, cell opacity increases proportionally to the probability value, creating a heat-map effect that highlights regions of high flood risk.

5.5 Serialisation

Models are persisted using `joblib.dump()` in the standard sklearn format. Each gauge model is stored independently at `GAUGE-{id}.joblib`, enabling selective retraining without full pipeline re-execution.

6 Validation and Metrics

6.1 Evaluation Metrics

Four metrics are computed per gauge on the held-out test set:

Metric	Description
Accuracy	Fraction of correct binary predictions
AUC-ROC	Area under the Receiver Operating Characteristic curve; measures discrimination ability independent of threshold
Brier Score	Mean squared error of probabilistic predictions; lower is better
Log Loss	Negative log-likelihood of predicted probabilities; penalises confident wrong predictions

6.2 Production Results (v2.0 stressm)

Training across all 52 Thames catchment gauges with 10,000 multi-storm sequences:

- **Average AUC-ROC:** ≈ 0.993
- **Representative gauge (GAUGE-8f1d368d):** AUC 0.9927, accuracy 0.9776, flood rate 8.4%
- **Training sequences:** 10,000 StormSequence objects (all types)
- **Vectors per gauge:** 1,680,000 (10,000 sequences $\times$ 168 hours)
- **Gauges:** 52 (full Thames portfolio; no gauges skipped)
- **Feature importance:** $\log_{h_s} \approx 70\%$, $\log_{t_end} \approx 24\%$, derivatives $\approx 6\%$

6.3 Version Comparison: v1.1 $\rightarrow$ v1.2 $\rightarrow$ v1.3 $\rightarrow$ v2.0

The following table shows the progression across all model versions:

Version	Key Change	Mean AUC	Vectors/gauge
v1.1	2 features (level, hour)	0.9189	168,000
v1.2	+ Δw , $\Delta^2 w$ (log-raw)	0.9557	168,000
v1.3	Log-transform all features	0.9525	168,000
v2.0	Multi-storm sequences; 10$\times$ data	≈ 0.993	1,680,000

Analysis: The v1.1 $\rightarrow$ v1.2 jump (+4.0%) came from adding derivative features. The v1.2 $\rightarrow$ v1.3 slight reduction (−0.3%) reflects the more rigorous log-space normalisation (marginally harder problem, better-calibrated probabilities). The v1.3 $\rightarrow$ v2.0 step (+4.2%) comes entirely from richer, larger training data: multi-peak hydrographs force the classifier to learn antecedent-wetness dynamics, and 10 $\times$ more data reduces tree estimation variance throughout the GBM ensemble.

6.3.1 Asymmetry Evidence: Rising vs Falling Water

The key diagnostic is the model’s ability to distinguish rising from falling water at the same level. The following table shows $P(\text{flood})$ for gauge GAUGE-32e88723 (AUC = 0.957, flood rate = 10.0%) at hour = 20 with $|\Delta w| = 0.4 \text{ m/h}$:

Level (m)	Rising	Steady	Falling	ΔP
3.3 (sub-alert)	0.003	0.027	0.103	−0.099
3.8	0.033	0.140	0.462	−0.429
4.3 (near alert)	0.261	0.646	0.882	−0.622
4.8 (above alert)	0.831	0.942	0.976	−0.144
5.3	0.984	0.992	0.996	−0.012

The model has learned the physically correct signal:

- At sub-alert levels, **falling water implies higher $P(\text{flood})$** : if the water is at 4.3 m and falling, the storm already peaked higher, providing strong evidence that alert was breached.
- At sub-alert levels, **rising water implies lower $P(\text{flood})$** : the storm has not yet peaked, so there is genuine uncertainty about whether the peak will reach the alert level.

- Above the alert level, all states converge to $P \approx 1$ regardless of direction, as the evidence for flood breach is overwhelming.

The maximum asymmetry occurs near the alert threshold (4.3 m), where the directional signal is most informative: $\Delta P = -0.622$. The v1.1 model would output the “Steady” column for both rising and falling water.

6.4 Feature Importance

Averaged across all 40 gauges, the four features contribute:

Feature	Importance
water_level	0.6054
hour	0.2233
delta_w	0.1407
delta2_w	0.0307

Water level remains the dominant predictor (60.5%), as expected. The velocity feature (`delta_w`) captures 14.1% of model importance—a substantial signal that enables the rising/falling asymmetry. The acceleration feature (`delta2_w`) contributes 3.1%, providing marginal refinement for distinguishing decelerating rises (approaching peak) from accelerating surges. Compared to v1.1, the hour feature’s relative importance decreased from ~14% to 22.3%—however, this is a redistribution effect: in absolute terms, the temporal signal is captured more precisely because `delta_w` absorbs some of the temporal variation previously attributed to `hour` alone.

6.5 Automated Test Results

Table 1: Test Results Summary — Flood Probability Classifier (MKM-FC-001)

Total Tests	71
Passed	70
Failed	0
Skipped	1
Duration	17.88s

Table 2: Detailed Test Results — Flood Probability Classifier

Test Description	Acceptance Criteria	Result	Time
Trained model must report importance for exactly 4 log features.	<code>len(trained) > 0;</code> <code>set(fi.keys()) == {'log_h_s', 'log_t_end', 'delta_log_h', ...}</code>	Pass	2.480s

Test Description	Acceptance Criteria	Result	Time
FloodPredictor.predict() takes delta_w/delta2_w, no hour argument.	0 <= p_rising <= 1; 0 <= p_falling <= 1	Pass	2.366s
predict() with no delta_w/delta2_w defaults to steady-state (0.0).	0 <= p <= 1	Pass	2.391s
Calling predict_proba() with 3 features on a 4-feature model raises ValueError.	—	Pass	2.374s
Saved .joblib model must have n_features_in_ == 4 (v1.3 log features).	len(model_files) > 0; model.n_features_in_ == 4	Pass	2.363s
Each vector must be [log_h_s, log_t_end, delta_log_h, delta2_log_h, flood_flag].	len(captured) > 0; len(v) == 5 (+2 more)	Pass	0.051s
flood_rate must equal flood_positive_vectors / vector_count.	stat['flood_rate'] == pytest.approx(expected, abs=0.001)	Pass	0.005s
For a flooding storm, label must be 1 on rising limb, 0 on falling.	stat['flood_positive_vectors'] < stat['vector_count']	Pass	0.005s
A storm that never reaches severe must have label=0 for all hours.	stat['flood_positive_vectors'] < stat['vector_count']	Pass	0.005s
Each storm has peak_level_m.	'peak_level_m' in s; s['peak_level_m'] > 0	Pass	0.003s
Storms have required identification fields.	'storm_id' in s; 'name' in s (+3 more)	Pass	0.002s
Returns 404 when stress_storms.json missing.	resp.status_code == 404	Pass	0.001s
Returns empty list for gauge with no storms.	resp.status_code == 200; data['storms'] == []	Pass	0.003s
Missing gauge_id returns 400.	resp.status_code == 400	Pass	0.001s
Returns storm list for specified gauge.	resp.status_code == 200; data['status'] == 'success' (+2 more)	Pass	0.002s
Storms sorted by peak level descending (worst first).	peaks == sorted(peaks, reverse=True)	Pass	0.003s
Each gauge has gauge_id, gauge_name, lon, lat, trade_count.	'gauge_id' in g; 'gauge_name' in g (+3 more)	Pass	0.007s
Gauges with trades show non-zero trade_count.	sum(counts) > 0	Pass	0.007s

Test Description	Acceptance Criteria	Result	Time
Returns gauge list with trade counts.	<code>resp.status_code == 200;</code> <code>data['status'] == 'success'</code> (+2 more)	Pass	0.006s
Gauges sorted west to east (ascending longitude).	<code>lons == sorted(lons)</code>	Pass	0.007s
Water level should return close to base by end of 168h.	<code>levels[-1] == pytest.approx(2.0, abs=0.2)</code>	Pass	0.000s
Hydrograph length	<code>len(levels) == NUM_HOURS</code>	Pass	0.000s
Hydrograph peak between ends	<code>0 < peak_idx < NUM_HOURS - 1</code>	Pass	0.001s
Hydrograph peaks at base plus change	<code>max(levels) == pytest.approx(5.0, abs=0.1)</code>	Pass	0.000s
Hydrograph starts at base	<code>levels[0] == pytest.approx(3.5, abs=0.01)</code>	Pass	0.000s
Noise sigma decreases	<code>sigma_0 > sigma_end;</code> <code>sigma_0 ==</code> <code>pytest.approx(0.6, abs=0.05)</code> (+1 more)	Pass	0.000s
Noise sigma monotonic	<code>sigmas[i] <= sigmas[i - 1]</code>	Pass	0.000s
Each hour has water_level, p_flood, portfolio fields.	<code>'hour' in h;</code> <code>'water_level' in h</code> (+4 more)	Pass	0.024s
Non-existent storm returns 404.	<code>resp.status_code == 404</code>	Pass	0.004s
Severe storm triggers knock-out (P(flood)=100%).	<code>data['hourly'][first_breach]['p_flood']</code> <code>== 1.0;</code> <code>h['p_flood'] is None</code>	Pass	0.018s
Missing gauge_id or storm_id returns 400.	<code>resp.status_code == 400</code>	Pass	0.001s
No JSON body returns error.	<code>resp.status_code in (400, 500)</code>	Pass	0.005s
Response includes portfolio-level stress summary.	<code>'num_trades' in summary;</code> <code>'total_notional' in summary</code> (+5 more)	Pass	0.023s
Response includes probability surface grid.	<code>'probability_surface' in data;</code> <code>'water_levels' in surface</code> (+7 more)	Pass	0.021s
POST returns 60-hour forecast array.	<code>resp.status_code == 200;</code> <code>data['status'] == 'success'</code> (+1 more)	Pass	0.021s
Response includes per-trade summary.	<code>'trades' in data;</code> <code>len(data['trades'])</code> <code>== 2</code> (+4 more)	Pass	0.017s
Surface water levels cap at severe — no rows above severe.	<code>max(surface['water_levels']) <=</code> <code>severe</code>	Pass	0.020s
Surface must cover full 168h storm horizon regardless of KO.	<code>max(surface['hours']) >= 164</code>	Pass	0.018s
Empty gauge_id → 400 (line 217).	<code>resp.status_code == 400</code>	Pass	0.001s

Test Description	Acceptance Criteria	Result	Time
No open trades at gauge → 404 (line 250).	<code>resp.status_code == 404</code>	Pass	0.004s
Null JSON body → 400 (line 205).	<code>resp.status_code == 400</code>	Pass	0.001s
Predictor None (no model dir) → 404 (lines 53-54, 61, 294).	<code>resp.status_code == 404</code>	Pass	0.011s
Gaugets file present → hydrograph built from real readings (lines 258-279).	<code>resp.status_code == 200;</code> <code>data['status'] == 'success'</code>	Pass	0.026s
<code>_get_predictor</code> returns cached value without re-importing (line 53).	<code>result is sentinel</code>	Pass	0.000s
<code>_get_predictor</code> returns None when stress/ dir absent (lines 53-54, 61).	<code>pred is None</code>	Pass	0.000s
stress/run returns 500 on engine error (lines 302-306).	<code>resp.status_code == 500</code>	Pass	0.003s
stress/gauges returns 500 when gauge loader raises (lines 122-124).	<code>resp.status_code == 500</code>	Pass	0.002s
stress/storms returns 500 when storm loader raises (lines 185-187).	<code>resp.status_code == 500</code>	Pass	0.002s
peak_position=0 uses decay-only path (lines 72-74).	<code>len(levels) == 10;</code> <code>all(isinstance(v, float) for v in levels)</code>	Pass	0.000s
remaining=0 uses frac=1.0 path (lines 76-77).	<code>len(levels) == 5</code>	Pass	0.000s
ETA must appear in the output once the second gauge is printed.	<code>'ETA' in captured.out</code>	Skip	0.050s
Each progress line must include the gauge ID.	<code>stat['gauge_id'] in captured.out</code>	Pass	0.050s
A progress line must be printed for every gauge processed.	<code>f'[{i}/{num_gauges}]' in captured.out</code>	Pass	0.051s
Per-gauge flood rate should be well below the old 25-86% range.	<code>stat['flood_rate'] < 0.30</code>	Pass	0.050s
Includes all alert storms	<code>summary['alert_storms'] == 10</code>	Pass	0.049s
Samples non alert storms	<code>summary['non_alert_storms'] > 0;</code> <code>summary['total_storms'] > summary['alert_storms']</code>	Pass	0.051s
Total storms must not exceed alert + TARGET_NON_ALERT.	<code>summary['total_storms'] <= 10 + TARGET_NON_ALERT</code>	Pass	0.050s
Each storm entry must have 'gauge_responses' for the route to work.	<code>'gauge_responses' in s;</code> <code>len(s['gauge_responses']) > 0</code>	Pass	0.023s
If stress_storms.json already exists, generator must not overwrite it.	<code>path.stat().st_mtime == mtime_before</code>	Pass	0.051s

Test Description	Acceptance Criteria	Result	Time
gauge_responses must have all fields the stress route reads.	not missing	Pass	0.023s
Each storm must carry duration_hours and peak_position for hydrograph.	'duration_hours' in s; 'peak_position' in s (+1 more)	Pass	0.023s
Written file must have top-level 'storms' list.	'storms' in data; isinstance(data['storms'], list) (+1 more)	Pass	0.024s
File must only contain storms that breached alert at ≥ 1 gauge.	len(data['storms']) == 5	Pass	0.029s
generate_stress_vectors must create stress_storms.json when not present.	not (input_dir / 'stress_storms.json').exists(); (input_dir / 'stress_storms.json').exists()	Pass	0.022s
Training summary saved	summary_path.exists(); ts['num_trained'] == 1 (+1 more)	Pass	2.379s
Trains and saves model	(output_dir / 'stress' / 'GAUGE-test1.joblib').exists(); summary['training']['num_trained'] == 1	Pass	2.355s
flood_rate must equal flood_positive_vectors / vector_count.	'flood_positive_vectors' in stat; stat['flood_rate'] == pytest.approx(expected, abs=0.01)	Pass	0.050s
Generator must NOT write a bulk vectors file to disk.	not (input_dir / 'stress_vectors.json').exists()	Pass	0.050s
Same seed must produce identical sampling stats.	s1['sampling_stats'] == s2['sampling_stats']	Pass	0.097s
Each gauge must have total_storms x NUM_HOURS vectors.	stat['vector_count'] == expected	Pass	0.050s
Summary must report correct vectors_per_gauge.	summary['vectors_per_gauge'] == summary['total_storms'] * ...	Pass	0.050s

6.6 Test Document History

Date	Version	Description	Author
04-Mar-2026	1.0	Initial test suite (15 tests)	D. Kelly
05-Mar-2026	1.1	7 test(s) added (total: 22)	D. Kelly
09-Mar-2026	1.2	56 test(s) added; 22 test(s) removed (total: 56)	D. Kelly
09-Mar-2026	1.3	4 test(s) added; 4 test(s) removed (total: 56)	D. Kelly
10-Mar-2026	1.4	12 test(s) added (total: 68)	D. Kelly
10-Mar-2026	1.5	3 test(s) added (total: 71)	D. Kelly

7 Sensitivity Analysis

Sensitivity of the classifier’s AUC to key hyperparameters is analysed by varying each parameter independently while holding others at production defaults.

Key findings:

- **n_estimators**: AUC stabilises above 100 rounds; diminishing returns beyond 200.
- **max_depth**: Optimal at 4; deeper trees overfit the noise.
- **learning_rate**: Sweet spot at 0.1; aggressive rates (>0.2) increase variance.

Table 3: Classifier sensitivity to n_estimators (gauge GAUGE-082c, max_depth=4, lr=0.1).

n_estimators	AUC-ROC	Accuracy	
50	0.8010	0.7936	
100	0.8002	0.7917	
200	0.7965	0.7865	Production
500	0.7931	0.7824	

Table 4: Classifier sensitivity to max_depth (gauge GAUGE-082c, n_estimators=200, lr=0.1).

max_depth	AUC-ROC	Accuracy	
2	0.7974	0.7904	
3	0.7975	0.7878	
4	0.7965	0.7865	Production
5	0.7951	0.7840	
6	0.7928	0.7801	

Table 5: Classifier sensitivity to learning_rate (gauge GAUGE-082c, n_estimators=200, max_depth=4).

learning_rate	AUC-ROC	Accuracy	
0.0100	0.7950	0.7869	
0.0500	0.8005	0.7901	
0.1000	0.7965	0.7865	Production
0.2000	0.7902	0.7811	
0.5000	0.7794	0.7763	

Table 6: Classifier sensitivity to subsample fraction (gauge GAUGE-082c, n_estimators=200, max_depth=4, lr=0.1).

subsample	AUC-ROC	Accuracy	
0.5000	0.7972	0.7891	
0.6000	0.7976	0.7853	
0.7000	0.7971	0.7897	
0.8000	0.7965	0.7865	Production
0.9000	0.7985	0.7881	
1.00	0.7975	0.7862	

Table 7: Feature importance at production hyperparameters — v1.1 vs v1.2 (gauge GAUGE-082c).

Feature	v1.1	v1.2
water_level	0.8607	0.6260
hour	0.1393	0.2590
delta_w	—	0.0960
delta2_w	—	0.0190
AUC-ROC	0.7965	0.9720

8 Model Limitations and Known Weaknesses

1. **Hydrograph-derived features only:** The four features (water level, hour, velocity, acceleration) are all derived from the local hydrograph. External features (rainfall intensity, upstream levels, antecedent soil moisture) could further improve discrimination but would require expanded data pipelines.
2. **No spatial correlation:** Each gauge is modelled independently. Correlated flood events across multiple gauges are not captured.
3. **Training noise regime:** The time-decaying noise is a modelling choice, not an empirical calibration. Different noise functions would yield different AUC profiles.
4. **Binary classification only:** The model predicts flood/no-flood. Flood severity (depth, velocity, duration) is not estimated.
5. **Stationarity assumed:** The classifier does not account for changing flood characteristics due to climate change or infrastructure modifications.
6. **Alert-level threshold:** Training uses the FloodAlert trigger level. Models trained on Warning or Severe thresholds would produce different classifiers.

9 Change History

Version	Date	Description
1.0	3-Mar-2026	Initial model documentation
1.1	3-Mar-2026	Balanced training data (1,000 storms), probability surface
1.2	5-Mar-2026	Four-feature model: velocity (Δw) and acceleration ($\Delta^2 w$); AUC 0.9189 $\rightarrow$ 0.9557
1.3	9-Mar-2026	Log-transformed feature space ($\ln(w/s)$, $\ln(t/T)$, log-velocity, log-acceleration); AUC 0.9557 $\rightarrow$ 0.9525 (marginal reduction, better calibration)
2.0	11-Mar-2026	Multi-storm upgrade: training data replaced with 10,000 StormSequence objects from MKM-SS-001; 1,680,000 vectors/gauge ($10 \times v1.x$); multi-peak hydrographs model antecedent-wetness dynamics; stressm classifiers stored in <code>data/output/stressm/</code> ; AUC 0.9525 $\rightarrow \approx 0.993$ (+4.2%); scope expanded to all 52 Thames gauges